SDC HACKATHON 2026

Phase 2: Final Project & Prototype Report

Hosted by Xiamen University Malaysia (XMUM) AI Club
Submission Deadline: 1 November 2026

Instructions: Complete every field below and submit as a single PDF document named "SDC2026_TeamName_Final.pdf" together with your pitch deck and a working deployment link. Answers must comply with the word limits stated for each open-ended question. Section 3 requires concrete, verifiable evidence. Unsupported claims will not be credited by judges.

1. Team & Project Overview

Team Name	"Hack into Google, No mistake"
Project / Product Name	SOPBox: Smart Receiving Check for F&B SMEs
Final Live Deployment URL	https://sopbox.replit.app
Codebase Repository URL	https://www.github.com/Dellrall/SOPBox
Name (Leader)	Lye Wei Lun
Email & phone (Leader)	dellralllye@gmail.com & 018-960 6378

2. Project Description

[Maximum 250 words]

SOPBox: Smart Receiving Check for F&B SMEs is an AI-powered platform designed to help small food and beverage businesses simplify their daily receiving operations, reduce manual work, and improve accuracy.

Many F&B SMEs still rely on paper delivery orders, spreadsheets, and manual checking when receiving goods. This process can be time-consuming and may lead to mistakes such as incorrect quantities, unsuitable storage temperatures, or missed delivery issues, which could eventually contribute to food waste.

To address these challenges, SOPBox allows staff to upload or take a photo of a delivery order, where AI automatically extracts the product information. Staff can then verify the details before the system checks quantities, temperatures, and other receiving requirements based on predefined rules. If any issues are detected, the delivery is flagged for manager review and approval.

Besides receiving checks, SOPBox also provides product and SOP management, AI-assisted SOP generation, inventory tracking, expiry monitoring, purchase order management, notifications, and audit logs. Different user roles and separate business workspaces help ensure that each business can manage its operations securely and independently.

By combining AI automation with human decision-making, SOPBox aims to make receiving operations faster, more consistent, and easier to manage. It also supports SDG 8.2 by improving SME productivity and SDG 12.3 by helping businesses reduce potential food losses.

3. Upgraded Product & Technical Execution

3.1 Final System Architecture

Provide a detailed description (and, if possible, an attached technical diagram) of your fully built system. Explain how your application integrates GenAI APIs, database structures, and backend/frontend frameworks into one unified solution.

SOPBox uses a multi-layer architecture that integrates the frontend, backend, database and external AI services into one platform. Each component has a specific responsibility, allowing the system to automate information processing while keeping important business decisions under human control.

Frontend Layer:

The frontend is developed using React 18, TypeScript, Vite and Tailwind CSS. It provides a responsive interface for staff, managers and owners to access receiving operations, product management, stock records, expiry tracking, approvals, reports and other business functions. The interface is designed for use on mobile phones, tablets and desktop computers.

Backend Layer:

The backend is developed using Python and FastAPI. It handles API requests, authentication, product management, delivery processing, stock updates, validation rules, approval workflows, reporting and activity logging. Business calculations and inventory decisions are processed by backend logic rather than relying on AI-generated answers.

AI Integration:

SOPBox uses Qwen3-VL-8B-Instruct (FP8), deployed through RunPod Serverless with vLLM, to read delivery-order images and assist with SOP generation. An additional OpenAI-compatible AI service supports sales-report extraction and the AI assistant. Extracted information is validated and presented for human review before it is used in business workflows.

Database Layer:

PostgreSQL, hosted through Replit, stores workspace information, user accounts, products, suppliers, deliveries, stock levels, expiry lots, sales, purchase orders, approvals and audit records. Row Level Security (RLS) helps enforce separation between business workspaces.

Rules and Approval Layer:

The backend applies predefined rules to check receiving temperatures, product quantities, packaging conversions and purchase-order balances. When discrepancies occur, affected delivery items can be held for review by a manager or business owner. Both managers and business owners can review the failed items and decide whether to accept, reject or return them. They can also manage the relevant inventory and operational workflows according to their assigned access permissions. All actions are recorded in the system's activity log.

Deployment and Integration:

The application is deployed through Replit Autoscale, while RunPod Serverless provides GPU-based AI processing. Telegram Bot API supports notifications and operational communication. The modular API structure also provides a foundation for future integration with point-of-sale systems and other SME platforms.

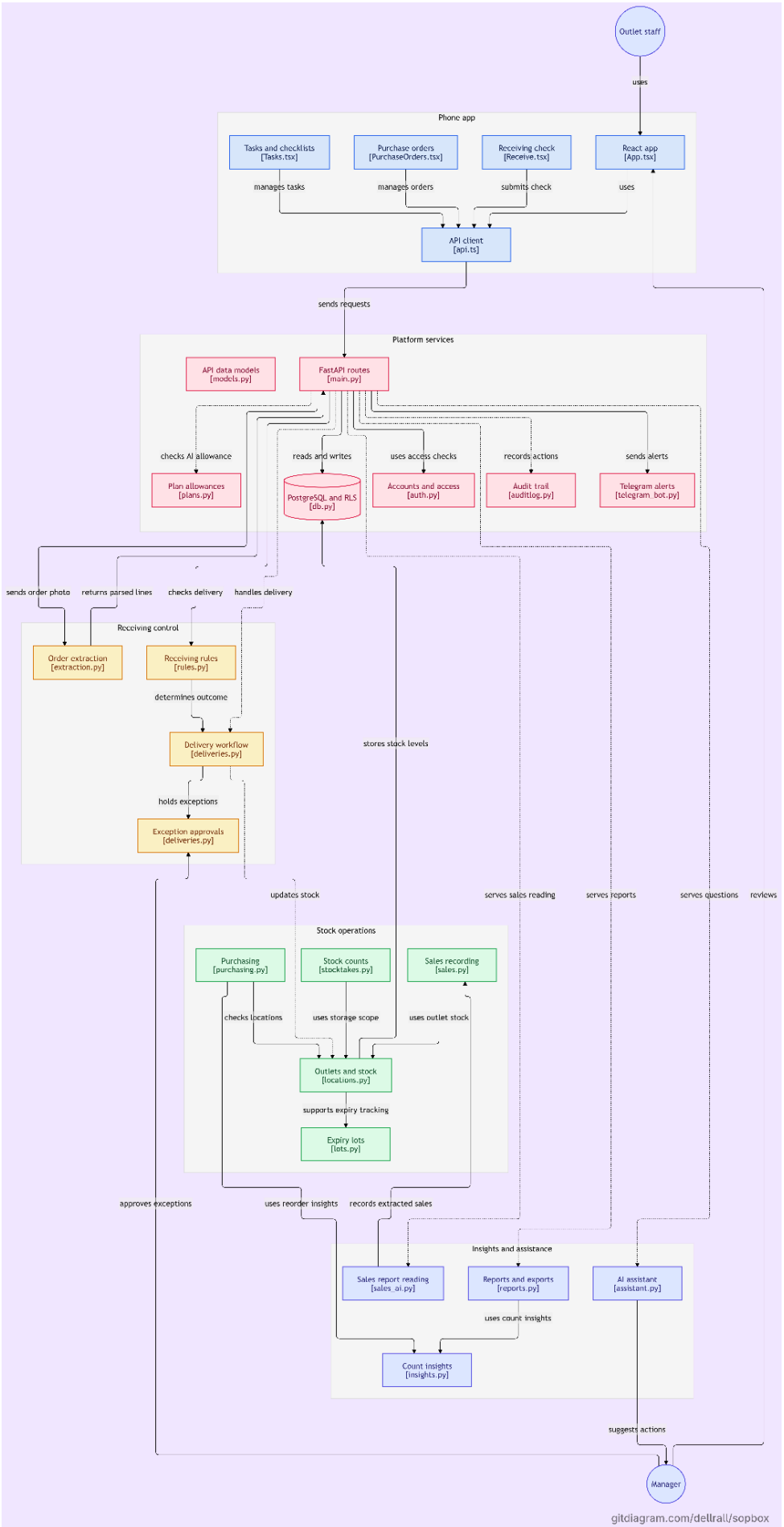

Outlet staff
uses
Phone app
Tasks and checklists [Tasks.tsx]
Purchase orders [PurchaseOrders.tsx]
Receiving check [Receive.tsx]
React app [App.tsx]
manages tasks
manages orders
submits check
uses
API client [api.ts]
sends requests
Platform services
API data models [models.py]
FastAPI routes [main.py]
checks AI allowance
reads and writes
uses access checks
records actions
sends alerts
Plan allowances [plans.py]
PostgreSQL and RLS [db.py]
Accounts and access [auth.py]
Audit trail [auditlog.py]
Telegram alerts [telegram_bot.py]
sends order photo
returns parsed lines
checks delivery
handles delivery
Receiving control
Order extraction [extraction.py]
Receiving rules [rules.py]
determines outcome
Delivery workflow [deliveries.py]
holds exceptions
Exception approvals [deliveries.py]
stores stock levels
updates stock
serves sales reading
serves reports
serves questions
reviews
Stock operations
Purchasing [purchasing.py]
Stock counts [stocktakes.py]
Sales recording [sales.py]
checks locations
uses storage scope
uses outlet stock
Outlets and stock [locations.py]
supports expiry tracking
Expiry lots [lots.py]
approves exceptions
uses reorder insights
records extracted sales
Insights and assistance
Sales report reading [sales_ai.py]
Reports and exports [reports.py]
AI assistant [assistant.py]
uses count insights
Count insights [insights.py]
suggests actions
Manager
gitdiagram.com/dellrall/sopbox

Figure 3.1.1: Architecture Diagram for SOPBox

3.2 Implementation & Hackathon Deliverables

Describe what was successfully built, integrated, and deployed during the 48-hour physical sprint. Highlight any specific problem-solving steps your team took to pivot or improve the project from the Phase 1 preliminary design.

[Maximum 300 words]

During the sprint we managed to get together and actually fixed some of the issues that were brought by 2 stays of non-stop development of Business feasibility, extra AI features like AI Chatbot, smart Sales reading, AI orchestrator, and more. Added API for external service integration and usage for premium user usage.

Some UI changes and utility modification and addition was made in order to make the app more usable and less bloaty with the side-menu compaction, home page compaction, mobile control revamp, setting page revamp, navigation revamp, documentation that are included in main page with privacy disclaimer, usage docs, and pricing page.

Fixes are also done to previously not working features such as Telegram notification and made structured AI development that understand the system better with AGENTS.md and CLAUDE.md. Additional model usage to optimize reading, and adding external flagship AI from OpenAI to enhance user experience.

4. Environmental & SDG Impact Verification

Validate the real-world impact of your built solution. Show concrete data that validate your environmental or social value proposition, compared against the baseline and KPI defined in previous phase submission.

*The data and evidence you provide in this section will be used to assess **SDG Impact, Technical Execution and Innovation scores.** Vague claims or missing verification will weaken your score across all three categories.*

Metric Category	**Your Response**
Target SDG Metric Achievement *Direct comparison between your Phase 1 baseline and your Phase 2 finalized prototype numbers* *e.g., "Achieved a 24.5% reduction in computational execution overhead against the 20% initial target."*	SOPBox supports SDG 8.2 by helping F&B SMEs reduce repetitive receiving tasks and SDG 12.3 by identifying delivery problems that could contribute to food waste. During Phase 1, our team proposed testing 20 simulated deliveries, including 5 with intentional discrepancies. We set targets of at least 90% discrepancy detection and a 30% reduction in receiving time. For Phase 2, we reviewed six delivery records from our prototype. Four deliveries passed all validation checks, while two contained identified problems. Delivery #30 showed a quantity mismatch and missing temperature information, while Delivery #29 showed a missing temperature reading for an item that was returned to the supplier. These results demonstrate that SOPBox can identify receiving issues and support appropriate follow-up actions. Although these records provide evidence of the system's functionality, they do not establish an overall detection accuracy or receiving-time reduction. Further controlled testing is needed to confirm whether the Phase 1 targets have been achieved. Based on our business impact model, a café spending RM20,000 monthly on stock could potentially save RM520 per month under the assumed waste-reduction

Metric Category	Your Response
	scenario. This is a projected benefit rather than a measured saving.
Verification Evidence Method *How was this data verified?* *e.g., codebase algorithmic profiling logs, hardware power-meter simulation readings, or database transaction counters.*	We verified the receiving validation and approval workflow by reviewing six delivery records generated in SOPBox. The records included four deliveries that passed all checks and two deliveries with identified problems. For Delivery #30, the system displayed a quantity mismatch and missing temperature information. The delivery was flagged for review and subsequently recorded as returned to the supplier, together with the decision-maker, timestamp, and reason. For Delivery #29, one product passed validation while another was returned due to damaged goods, with a missing temperature reading also displayed. The system retained the return decision and supporting photographic evidence. Screenshots of these records provide evidence of discrepancy identification, human decision-making, and traceability. The system also maintains receiving and approval records for future reference.
Socio-Economic / Civic Value *How does your deployed application translate into a scalable, real-world solution capable of being supported by civic or corporate partners?*	SOPBox helps small F&B businesses manage receiving operations more consistently without requiring expensive or complicated systems. AI-assisted extraction reduces repetitive data entry, while validation rules and managerial approval help staff identify problems before goods enter inventory. By reducing receiving errors and improving stock visibility, SOPBox may help businesses avoid unnecessary purchases and potential food losses. Its multi-workspace structure also allows restaurants, cafés, bakeries, and catering businesses to manage their own products, receiving rules, and staff independently.

Metric Category	Your Response
	The platform could be introduced through SME digitalization programmes, F&B associations, or partnerships with technology providers.
Deployment & Commercial Feasibility *e.g., modeled as an API service plugin designed for easy integration into existing corporate or institutional systems.*	SOPBox is deployed through Replit Autoscale, with PostgreSQL for data storage and RunPod Serverless for AI processing. This cloud-based structure allows businesses to access the platform without maintaining their own servers. The application supports multiple workspaces, role-based permissions, and configurable business rules, making it suitable for different SMEs. Its modular backend also provides opportunities for future integration with POS systems and other business platforms. A freemium pricing model allows smaller businesses to try the basic features before upgrading to paid plans for additional AI usage and advanced functions.

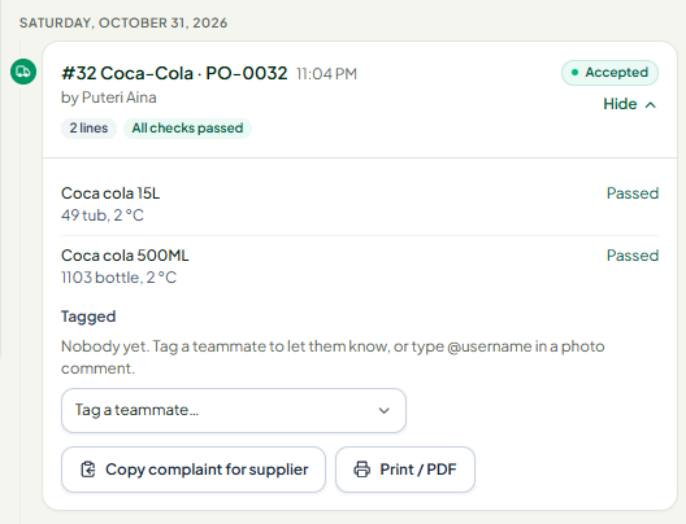


Figure 4.0.1: Successful Validation #32

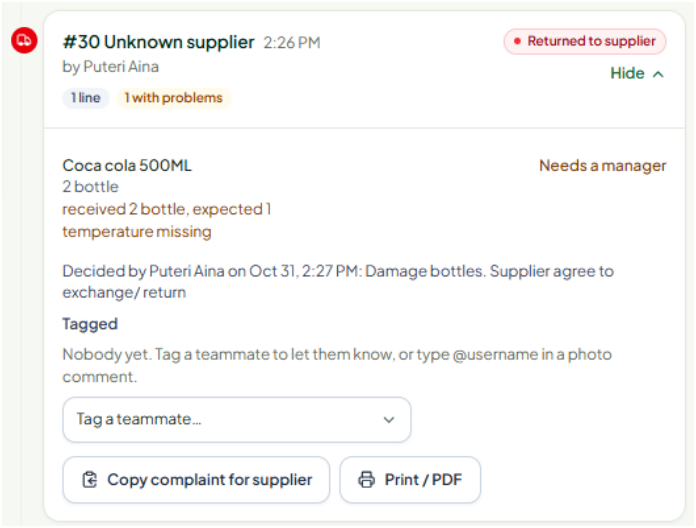


Figure 4.0.2: Discrepancy detection and decision #30

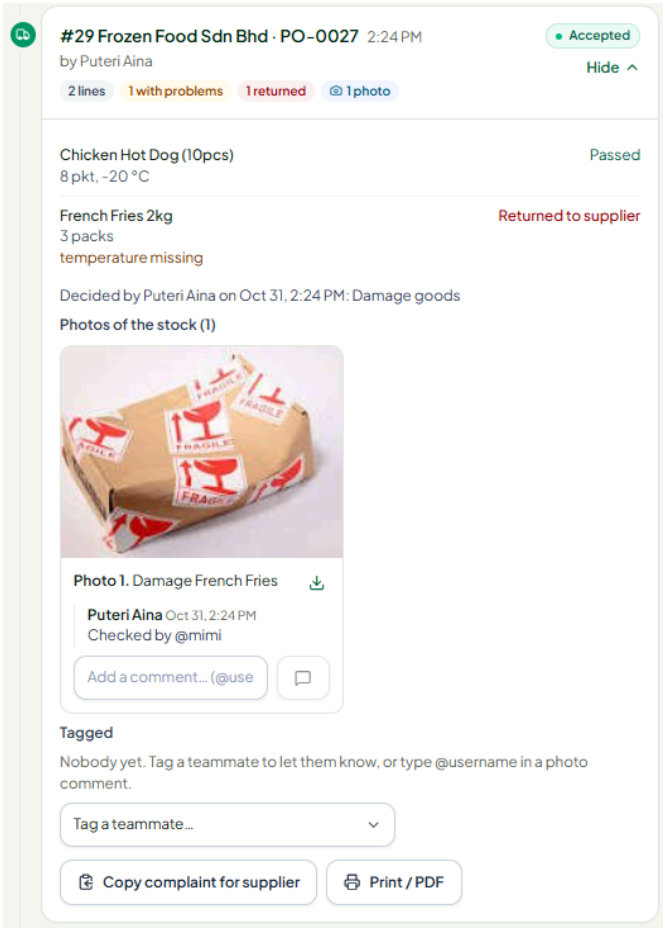


Figure 4.0.3: Partial acceptance and photographic evidence #29

4.1 Mathematical Proof / Code Logic Snippet (Optional, but holds bonus marks)

Paste the specific code section, mathematical formula, simulation log output, or calculation logic that proves the impact claims made above.

This proof is graded across three categories at once: ***Impact and Feasibility*** *(does the math support your claim),* ***Technical Execution*** *(does it reflect what you actually built), as well as* ***Innovation and Creativity*** *(is the approach original).] Do not leave this section generic or copied from Phase 1.*

Order the right amount, so less over-buying spoils (insights.py, trend and reorder): a straight line is fitted to 8 weeks of sales and kept between half and double the last 4 weeks' average, so one odd week cannot distort an order.

slope = $\Sigma(x-\bar{x})(y-\bar{y}) / \Sigma(x-\bar{x})^2$

forecast = clamp($\bar{y}$ + slope·(n − $\bar{x}$), recent/2, recent×2)

need = ceil(daily_use × 7 − flat_sales + forecast) + min_stock − stock − already_on_order

== 5. Impact formulas (docs/IMPACT.md), cafe buying RM20,000 a month ==

cafe 10%: waste RM2,000/month -> saving RM520/month = RM6,240/year

high 23%: waste RM4,600/month -> saving RM1,196/month = RM14,352/year

time: 60 deliveries x (6 - 1.5) min / 60 = 4.5 h/month x RM8.17 = RM36.77/month

5. Market Viability & Future Scalability Plan

SDC aims to encourage solutions that transition from innovations into market-ready applications. If given access to corporate funding, industry grants, or partnership opportunities (e.g., through SDC's Rotary network or joint research initiatives at XMUM), how would your team scale this prototype to process real-world data at volume?

[Maximum 250 words]

The current model and scalability are built upon with an internal backend that can manage the service better. With more workspace and business using the service through the free introduction of our system, it allow user to experience our application without paying and only when they feel like lesson their load of work with AI, they can do. Our plans uses flexible pricing which we market users with discounted usage pricing.

We felt that the price that we set is fair when compared to other similar applications within the market, which there isn't much to begin with. Our features focus on SOP crafting, rule setting, reading, and monitoring. This is a very niche market but one that the market can do to get some assistance with added value that our application can make.

We plan to scale the app as we go starting on focusing on profiting and usage hungry features, and going to a more marketing heavy platform to get our services known.

6. Final Submission Checklist

Tick all boxes that apply before submitting.

- [/] The application is fully compiled, live, and accessible via the deployment link provided in Section 1.
- [/] The GitHub (or equivalent) repository contains a clean, well-documented README with installation instructions.
- [/] The GitHub (or equivalent) repository shows the history of incremental code updates made throughout the official live hacking window.
- [/] The final pitch deck has been exported and saved as a flattened PDF file.
- [/] All fields in this template have been completed and word limits respected.

7. IP & Originality Declaration

7.1 Pre-Existing / Prior-Owned Work

Does this submission include any code, design, research, or other work that any team member created or owned before the start of this hackathon?

☐ Yes ☐ No ☑ No change from Phase 1 declaration

If yes, briefly describe what was pre-existing and which team member(s) owned it:

7.2 Third-Party Code, Datasets, or Models

Does your project use any third-party code libraries, open-source frameworks, public/private datasets, or pre-trained AI models that your team did not create?

☑ Yes ☐ No ☐ No change from Phase 1 declaration

If yes, list them below (open-source use is permitted provided it is declared and properly licensed/attributed):

Name	Type (code/dataset/model)	License
React / React DOM	**Code (Frontend UI framework)**	**MIT**
TypeScript	**Code (Programming language)**	**Apache-2.0**
Vite / React Plugin	**Code (Frontend build tools)**	**MIT**
Tailwind CSS	**Code (UI styling framework)**	**MIT**
Lucide React	**Code (Icon library)**	**ISC**
FastAPI	**Code (Backend web framework)**	**MIT**
Uvicorn	**Code (Python web server)**	**BSD-3-Clause**
Pydantic	**Code (Data validation)**	**MIT**
PostgreSQL	**Code (Database management)**	**PostgreSQL License**

psycopg / psycopg-pool	**Code (PostgreSQL connectivity)**	**LGPL-3.0**
HTTPX	**Code (External API communication)**	**BSD-3-Clause**
openpyxl	**Code (Excel spreadsheet processing)**	**MIT**
WebAuthn libraries	**Code (Passkey authentication)**	**MIT / BSD-3-Clause**
Qwen3-VL-8B-Instruct -FP8	**Model (AI image and document processing)**	**Apache-2.0**
vLLM	**Code (AI model inference engine)**	**Apache-2.0**
RunPod Serverless	**Service (AI model hosting)**	**Commercial service**
GPT-5.4-mini via KrackedDev	**Model / Service (AI assistant and sales processing)**	**Proprietary / Paid service**
Replit Autoscale	**Service (Application hosting)**	**Commercial service**
Telegram Bot API	**Service (Notifications)**	**Telegram Terms of Service**
Google Sign-In	**Service (User authentication)**	**Google Terms of Service**
Orbyte Template	**Code / Design (Frontend template)**	**License to be confirmed**

7.3 Employment-Related IP Conflicts

Is any team member currently bound by an employment, internship, or consulting agreement under which their employer could claim ownership of work they create during this hackathon?

☐ Yes ☐ No ☑ No change from Phase 1 declaration

If yes, please describe the nature of the agreement and which member it concerns:

8. Declaration

By submitting this report, the team leader confirms that the deployed application and data presented above are the original work of the team, completed within the official SDC 2026 hackathon period.



1/11/2026